

Fantasy NFL Game — Build Roadmap

A step-by-step process from where you are now to a finished, resume-ready project. Check off each step as you complete it. Don't skip ahead to Phase 3 before Phase 2 is solid — the whole point of this order is that each layer is fully working and understood before the next one depends on it.

Phase 0 — Environment & Repo (✅ Done)

- [x] GitHub repo created
 - [x] Local folder structure (`backend/app/domain`, `api`, `db`, `tests`, `notebooks`, `frontend`)
 - [x] `.gitignore` in place
 - [x] Python 3.12 virtual environment working
 - [x] Core packages installed: `fastapi`, `uvicorn`, `sqlalchemy`, `pytest`, `nfl_data_py`, `jupyter`, `pandas`
 - [x] First commit pushed to `main`
-

Phase 1 — Data Exploration (Jupyter)

Goal: understand exactly what data you have before designing classes around it.

[] In `notebooks/01_player_data_exploration.ipynb`, load a season of weekly data:
`import nfl_data_py as nflweekly = nfl.import_weekly_data([2024])`

- 1.
2. [] List all columns — identify which ones you actually need (`name`, `position`, `team`, `fantasy_points`, `fantasy_points_ppr`).
3. [] Check `weekly['position'].unique()` — confirm which positions are present. (Likely QB/RB/WR/TE only — no DST or K.)

[] Decide: **season totals or single week?** Try both:
`season_totals = weekly.groupby(['player_id', 'player_display_name', 'position', 'recent_team'])['fantasy_points_ppr'].sum().reset_index()`

- 4.

5. [] Check for messiness: duplicate names, players who changed teams mid-season, near-zero stat players (injured/inactive).
6. [] Figure out **D/ST data** — `nfl_data_py` doesn't include this cleanly. Options:
 - Compute team defense fantasy points yourself from `import_pbp_data` (points allowed, sacks, turnovers, etc.) — more work but accurate.
 - Use a small hardcoded/manual dataset for D/ST just to unblock development, revisit later.
7. [] Figure out **Kicker data** — check if `import_weekly_data` includes kicking stats, or if you need a separate pull.
8. [] Once the shape of clean data is clear, export it to a CSV in `backend/app/data/` (e.g. `players_2024.csv`) so your domain layer doesn't depend on a live library call every time you test.
9. [] Commit the notebook + exported CSV.

Definition of done: you have one CSV with columns roughly like `name`, `position`, `nfl_team`, `fantasy_points`, covering QB/RB/WR/TE/DST/K, that you trust.

Phase 2 — Domain Layer (pure Python, no frameworks)

Goal: the actual game, fully playable from a plain Python script — no web server, no database yet.

1. [] `player.py` — a `Player` dataclass: `id`, `name`, `position`, `nfl_team`, `fantasy_points`.
2. [] `roster.py` — a `Roster` class:
 - Holds the 9 slots (QB, RB1, RB2, WR1, WR2, TE, FLEX, DST, K) as a dict.
 - `eligible_slots(player)` → which open slots a player could fill.
 - `assign(player, slot)`.
 - `total_points()`.
 - `is_full()`.
3. [] `participant.py` — a `Participant` class: `id`, `name`, `is_ai`, `roster`.
4. [] `game_engine.py` — the turn-based engine:
 - `build_turn_order(participant_ids, num_rounds, snake: bool)` — the one function that differs between snake/non-snake modes.
 - `roll_team()` — random NFL team, returns still-available players from it.
 - `make_pick(participant, player_id, slot)`.
 - `make_ai_pick(participant)` — reuse for AI opponents *and* solo mode.
 - `advance_turn()`.
 - `is_complete()`.

- `standings()`.
- 5. [] Write a small `if __name__ == "__main__":` script in `backend/app/domain/` (or a scratch notebook) that plays a full 2-participant game start to finish, printing each pick — this is your proof the engine works before anything else touches it.

Definition of done: you can run a complete game in a plain Python script/REPL, with 2+ human or AI participants, snake or non-snake, and get a final standings list — with zero web framework or database involved.

Phase 3 — Tests (pytest)

Goal: lock in correctness before you build more layers on top.

1. [] `tests/test_roster.py` — slot eligibility rules, FLEX logic, bench overflow, `total_points()`.
2. [] `tests/test_game_engine.py`:
 - Snake order produces correct "double pick" behavior at round turns.
 - Non-snake order repeats identically each round.
 - A player, once picked, can never be picked again.
 - `make_ai_pick` always picks a legal, available player.
 - `is_complete()` triggers correctly when all rosters are full.
3. [] Run `pytest` from `backend/` and confirm everything passes.
4. [] Commit.

Definition of done: green test suite covering the core rules — this is what makes the project resume-credible (shows you test your logic, not just "it worked when I tried it").

Phase 4 — Persistence Layer (SQLAlchemy)

Goal: store real player data and in-progress/completed games.

1. [] `db/models.py` — SQLAlchemy models: `PlayerModel`, `GameModel`, `ParticipantModel`, `PickModel`.
2. [] A seed script (`backend/app/db/seed.py`) that loads your Phase 1 CSV into the `players` table.
3. [] Decide: SQLite for local dev (simple, file-based, zero setup) — fine for a resume project, don't over-engineer with Postgres unless you want the deployment complexity later.

4. [] Write functions to translate between DB rows and your Phase 2 domain objects (keep these separate — domain objects should stay framework-agnostic).
5. [] Test: seed the DB, load players into a domain-layer game, play it, confirm nothing broke.

Definition of done: player data and game state persist across runs, and your domain layer still doesn't know SQLAlchemy exists.

Phase 5 — API Layer (FastAPI)

Goal: expose the engine over HTTP so a frontend can drive it.

1. [] `api/routes_games.py`:
 - `POST /games` — create a game (participants, snake/non-snake, AI count).
 - `GET /games/{id}` — current state (whose turn, rosters, rolled team).
 - `POST /games/{id}/roll` — roll a team for the current turn.
 - `POST /games/{id}/pick` — make a pick.
 - `GET /games/{id}/standings`.
2. [] Wire FastAPI's dependency injection to hand routes a DB session.
3. [] Test every endpoint manually via the auto-generated docs at `/docs` (FastAPI gives you this for free).
4. [] Add basic error handling: invalid picks, picking an already-taken player, acting out of turn.

Definition of done: you can play a full game using nothing but `curl` or the `/docs` UI — no frontend needed yet.

Phase 6 — Frontend (React)

Goal: a playable UI.

1. [] Scaffold with Vite: `npm create vite@latest frontend -- --template react`.
2. [] Screens:
 - **Lobby** — set up a game: number of participants, human vs. AI, snake toggle.
 - **Draft board** — shows rolled team, available players from it, whose turn, all rosters filling in.
 - **Standings** — final rankings.

3. [] Connect to the FastAPI backend via `fetch`.
4. [] Poll or refetch state after each action (keep it simple before reaching for websockets).

Definition of done: you can play a full multi-participant game end-to-end in the browser.

Phase 7 — Game Modes

Layer these on top of the working core, one at a time:

- [] **2-player** — baseline, already covered by the engine design.
 - [] **3+ player** — confirm turn order and UI both scale (they should, since `participants` is just a list).
 - [] **Snake vs. non-snake toggle** — surfaced as a lobby option.
 - [] **AI opponents** — any participant can be flagged `is_ai`; engine auto-picks on their turn.
 - [] **Solo "high score" mode** — 1 human, 0 AI, just filling a roster and seeing your final point total (maybe track a local leaderboard of past solo runs).
-

Phase 8 — Polish for Resume-Worthiness

- [] README with a clear project description, screenshots/GIF of gameplay, and setup instructions.
 - [] `.env.example` documenting any config needed.
 - [] CI: a simple GitHub Actions workflow that runs `pytest` on every push.
 - [] Deploy: backend on something like Render/Fly.io, frontend on Vercel/Netlify — a live demo link matters a lot on a resume.
 - [] Write a short "Architecture" section in the README explaining the domain/api/db separation — this is the kind of thing interviewers actually ask about.
-

Where you are right now

You're at the start of **Phase 1**. Next concrete action: finish the `nfl_data_py` exploration in your notebook and get a clean player CSV exported.